Swinburne University of Technology Hawthorn Campus
Department of Computing Technologies

COS20028 Big Data Architecture and Application

Assignment 1 - *Semester 2, 2025*

Student Name: NGUYEN PHAM DUY
Student ID: 104058375

Assessment Title: MapReduce Application.
Assessment Outcome: 20%
Due Date: AEST 23:59 on 11/06/2025.
Assessable Item:

- One (1) piece of a report containing the complete flowchart showing your design idea about how to prepare the data, the tools being used in all steps, and the report for answers to the given questions.
- One (1) piece of a video explaining how you found answers to the question, the challenges you encountered, and how you solved them.

---

1. Correct implementation of the Mapper class(es) with readability and well-structured codes and comments. **[20 marks]**

"***WordMapper***" is the mapper for the first MapReduce job. It filters words from the dataset that begin with an uppercase letter from the set {A, C, G, W, Y} and are followed by lowercase letters. It uses the regular expression *"\\b[ACGWY][a-z]+\\b"* to perform the match and emits each valid word with a count of one.

*WordMapper.jav | WordCount.java | SumReducer.java | TopSumReducer.j | TopWordCount

```java
package stubs;

import java.io.IOException;

/**
 * WordMapper filters and emits words that:
 * - Start with one of the uppercase letters: "A,C,G,W,Y"
 * - Followed by lowercase letters
 *
 * For each matching word, this mapper emits (word, 1) as output
 *
 */
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    //Reusable Text object to avoid creating new objects in loop
    private Text word = new Text();

    //Regex Pattern
    private Pattern pattern = Pattern.compile("\\b[ACGWY][a-z]+\\b");

/**
   * The map method is called once per line of input.
   * It finds all words matching the defined pattern and emits them with count 1.
*/
  @Override
  public void map(LongWritable key, Text value, Context context)
```

```java
  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
      // Apply regex matcher to the line
      Matcher matcher = pattern.matcher(value.toString());
      // For each match, emit the word with count 1
      while (matcher.find()) {
        // Extract matched word
          word.set(matcher.group());
        // Emit (word, 1)
          context.write(word, new IntWritable(1));
      }
  }
}
```

Figure 1. Mapper class for Job 1 – Word Filtering.

"***TopWordMapper***" is used in the second MapReduce job. It reads the output from the first MapReduce job and switches the key-value pairs from (word, count) to (count, word), enabling sorting to arrange the counts in descending order by multiplying the key (count) by -1.

SumReducer.java | TopSumReducer.j | TopWordCount.ja | TopWordMapper.j | WordMapper.ja

```java
package stubs;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

/**
 * Processes output lines from the first MapReduce job
 * Each line is expected to contain: <word> <count>
 * This mapper parses each line, swaps the key and value to (count, word)
 */
public class TopWordMapper extends Mapper<LongWritable, Text, IntWritable, Text> {
    // Reusable Hadoop writable objects for key (count) and value (word)
    private IntWritable outKey = new IntWritable();
    private Text word = new Text();

    /**
     * The map method processes each line of the input file.
     * Each line should be: <word> <count>
     * It splits the line, converts the count to integer, and emits (count, word).
     */
  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
    // Split the line by whitespace
      String[] parts = value.toString().split("\\s+");
    // Ensure line has exactly two components: word and count
      if (parts.length == 2) {
          try {
              int count = Integer.parseInt(parts[1]);
              // Negate to reverse order by multiplying count with -1
              outKey.set(-count);
              word.set(parts[0]);
              context.write(outKey, word);
          } catch(NumberFormatException e) {
              // Ignore malformed lines where count is not a number
          }
      }
  }
}
```

Figure 2. Mapper class for Job 2 – Key-Value Swapping and Sorting Descending Order.

2. Correct implementation of the Reducer class(es) with readability and well-structured codes and comments. **[20 marks]**

"***SumReducer***" serves as the reducer in the first MapReduce job. It aggregates the values (counts) associated with each word emitted by the mapper and calculates the total frequency of each word.

*SumReducer.jav | TopSumReducer.j | TopWordCount.ja | TopWordMapper.j | WordMapper.java

```java
package stubs;

import java.io.IOException;

/**
 * SumReducer receives (word, list of counts) and sums them.
 *
 * This is used in the first job to count total occurrences of each word.
 */
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    /**
     * The reduce method is called once for each unique key (word).
     * It sums up all the integer values
     */
  @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
```

```java
  @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
        int wordCount = 0;

        // Sum all the values associated with this word
        for (IntWritable value : values) {

            wordCount += value.get();
        }
        // Emit the word along with its total count
        context.write(key, new IntWritable(wordCount));
    }
}

```

Figure 3. Reducer class for Job 1 – Word Count Aggregation.

"***TopSumReducer***" class as the reducer in the second job. It receives sorted (count, word) pairs, restores the original positive count, and outputs the two most frequent terms as well as the total number of words that appeared exactly once under the given criteria.

*SumReducer.jav | TopSumReducer.j | TopWordCount.ja | TopWordMapper.j | WordMapper.java | SumR

```java
package stubs;

import java.io.IOException;

/**
 * TopSumReducer processes (count, [words]) pairs.
 *
 * Perform two tasks:
 * 1. Outputs the top 2 most frequent words with their counts.
 * 2. Counts and reports how many words occurred exactly once (count == 1).
 */
public class TopSumReducer extends Reducer<IntWritable, Text, Text, IntWritable> {
     // Keeps track of how many top-ranked words have been output
    private int rank = 0;

    // Counter for words that appear only once in the dataset
    private int singleCountWords = 0;

    /**
     * Called once per unique count (key), with all words (values) that share that count.
     */
  @Override
    public void reduce(IntWritable key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
      // Negated count, restore the original positive count
```

```
*SumReducer.jav | TopSumReducer.j | TopWordCount.ja | TopWordMapper.j | WordMapper.java | SumReducer.
29      // Negated count, restore the original positive count
30      int count = -key.get();
31        for (Text word : values) {
32            // Output the top 2 words with the highest frequencies
33            if (rank < 2) {
34                rank++;
35                context.write(new Text ("Top" + rank + ": " + word.toString()), new IntWritable(count));
36            }
37
38            // Count how many words occurred exactly once
39            if (key.get() == -1) {
40                singleCountWords++;
41            }
42        }
43    }
44  /**
45   * Called once after all reduce calls are finished.
46   * Outputs the total number of words that occurred only once.
47   */
48  @Override
49  protected void cleanup(Context context) throws IOException, InterruptedException {
50      context.write(new Text("Single-count words"), new IntWritable(singleCountWords));
51  }
52 }
53
```

Figure 4. Reducer class for Job 2 – Extract Top Words and Single-Count Terms.

3. Correct implementation of the Driver class(es) with readability and well-structured codes and comments. **[20 marks]**

"***WordCount***" is the driver class for the first MapReduce job. It sets up the job configuration, invokes the "***WordMapper***" and "***SumReducer***" classes, and processes the dataset to produce the word frequency list based on the regex criteria.

```
WordCount.java | *SumReducer.jav | TopSumReducer.j | *TopWordCount.j | *TopWordMapper.
1  package stubs;
2
3  import org.apache.hadoop.conf.Configuration;
10
11 /**
12  * WordCount is the driver class for the first MapReduce job.
13  *
14  * This job performs word counting based on a regex pattern.
15  * It reads input text, uses WordMapper and SumReducer to compute word frequencies,
16  * and writes the output to the specified directory.
17  */
18 public class WordCount {
19
20   public static void main(String[] args) throws Exception {
21     // Check for exactly 2 command-line arguments: input path and output path
22     if (args.length != 2) {
23       System.out.printf(
24           "Usage: WordCount <input dir> <output dir>\n");
25       System.exit(-1);
26     }
27
28  // Create a new Hadoop configuration and job instance
29     Configuration conf = new Configuration();
30     Job job = Job.getInstance(conf, "Word Count");
31
```

WordCount.java | *SumReducer.jav | TopSumReducer.j | *TopWordCount.j | *TopWordMapper.

```
31
32  // Set the main class containing the job (used for JAR)
33     job.setJarByClass(WordCount.class);
34
35  // Specify input and output paths using arguments
36     FileInputFormat.setInputPaths(job, new Path(args[0]));
37     FileOutputFormat.setOutputPath(job, new Path(args[1]));
38
39  // Set the mapper and reducer classes
40     job.setMapperClass(WordMapper.class);
41     job.setReducerClass(SumReducer.class);
42
43  // Print author information
44     System.out.print("Programmer: Nguyen Pham Duy\nStudent ID: 104058375\n");
45
46  // Set the types for output key-value pairs
47     job.setOutputKeyClass(Text.class);
48     job.setOutputValueClass(IntWritable.class);
49
50     // Submit the job and exit based on success
51     boolean success = job.waitForCompletion(true);
52     System.exit(success ? 0 : 1);
53   }
54 }
55
```

Figure 5. Driver class for Job 1 – Execution setup for filtering and counting.

"***TopWordCount***" is the driver for the second MapReduce job. It reads the output from the first job, swaps the key-value pair and applies descending sort using "***TopWordMapper***", and invokes the reducer to produce the final top-2 and single-count word statistics.

TopSumReducer.j | *TopWordCount.j | TopWordMapper.j | WordMapper.java | SumReducer.java | WordC

```
1  package stubs;
2
3  import org.apache.hadoop.conf.Configuration;
10
11 /**
12  * TopWordCount is the driver class for the second MapReduce job.
13  *
14  * This job takes output from the first job (word count) as input.
15  * It reverses (word, count) to (count, word) and sorts results in descending order of count.
16  * The reducer then extracts the top 2 most frequent words and counts
17  * how many words occurred only once.
18  */
19 public class TopWordCount {
20
21   public static void main(String[] args) throws Exception {
22     // Ensure exactly 2 arguments are passed: input path and output path
23     if (args.length != 2) {
24       System.out.printf(
25           "Usage: TopWordCount <input dir> <output dir>\n");
26       System.exit(-1);
27     }
28
29     // Create configuration and job instance
30     Configuration conf = new Configuration();
31     Job job = Job.getInstance(conf, "Top Word Count");
```

TopSumReducer.j | TopWordCount.ja | TopWordMapper.j | WordMapper.java | SumReducer.java | WordC

```
28
29      // Create configuration and job instance
30      Configuration conf = new Configuration();
31      Job job = Job.getInstance(conf, "Top Word Count");
32
33  // Set the driver class (for JAR reference)
34      job.setJarByClass(TopWordCount.class);
35
36      // Set input and output paths from command-line arguments
37      FileInputFormat.setInputPaths(job, new Path(args[0]));
38      FileOutputFormat.setOutputPath(job, new Path(args[1]));
39
40      // Set the Mapper and Reducer classes
41      job.setMapperClass(TopWordMapper.class);
42      job.setReducerClass(TopSumReducer.class);
43
44      //Print author information
45      System.out.print("Programmer: Nguyen Pham Duy\nStudent ID: 104058375\n");
46
47  // Set intermediate output types from mapper
48      job.setMapOutputKeyClass(IntWritable.class);
49      job.setMapOutputValueClass(Text.class);
50
51  // Set final output types from reducer
52      job.setOutputKeyClass(Text.class);
```

```
50
51   // Set final output types from reducer
52      job.setOutputKeyClass(Text.class);
53      job.setOutputValueClass(IntWritable.class);
54
55   // Submit the job and exit based on success
56      boolean success = job.waitForCompletion(true);
57      System.exit(success ? 0 : 1);
58    }
59  }
60
```

Figure 6. Driver class for Job 2 – Setup for sorting and result extraction.

(*)Additional MapReduce Implementation: Counting Terms that start with all upper-case letters and follow by the lower-case letters Appearing Only Once in the Entire Document:

To handle the task of counting how many terms that start with all upper-case letters and follow by the lower-case letters appear only once in the entire dataset, an additional MapReduce workflow is implemented. This process uses two MapReduce jobs:

First Job: Word Frequency: The first job uses a mapper that emits all words that satisfy the regular expression *"\\b[A-Z][a-z]+\\b"*. The reducer then sums up the total frequency of each word.

*WordMapper.jav | SumReducer.java | WordCount.java | TopWordMapper.j | TopSumReduce

```java
package stubs;

import java.io.IOException;

/**
 * WordMapper filters and emits words that:
 * - Start with all of the uppercase letters
 * - Followed by lowercase letters
 *
 * For each matching word, this mapper emits (word, 1) as output
 *
 */
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    //Reusable Text object to avoid creating new objects in loop
    private Text word = new Text();
    //Regex Pattern
    private Pattern pattern = Pattern.compile("\\b[A-Z][a-z]+\\b");
/**
   * The map method is called once per line of input.
   * It finds all words matching the defined pattern and emits them with count 1.
*/
  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
      // Apply regex matcher to the line
```

```java
*/
  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
      // Apply regex matcher to the line
      Matcher matcher = pattern.matcher(value.toString());
      // For each match, emit the word with count 1
      while (matcher.find()) {
        // Extract matched word
          word.set(matcher.group());
        // Emit (word, 1)
          context.write(word, new IntWritable(1));
      }
  }
}
```

Figure 7. The mapper class for emitting every word in the document with a frequency 1.

*WordMapper.jav *SumReducer.jav WordCount.java TopWordMapper.j TopSumReducer.j TopWo

```java
package stubs;

import java.io.IOException;

/**
 * SumReducer receives (word, list of counts) and sums them.
 * This is used in the first job to count total occurrences of each word.
 */
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    /**
     * The reduce method is called once for each unique key (word).
     * It sums up all the integer values (which are typically 1s from the mapper).
     */
  @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
        int wordCount = 0;
        // Sum all the values associated with this word
        for (IntWritable value : values) {
            wordCount += value.get();
        }
        // Emit the word along with its total count
        context.write(key, new IntWritable(wordCount));
    }
}
```

Figure 8. The reducer class for aggregating word counts from the first job.

Second Job: Counting Single-Occurrence Terms: The second job reads the output of the first and emits (count, 1) pairs. The reducer filters for count == 1 and sums the total number of unique words that appeared exactly once.

*WordMapper.jav *SumReducer.jav WordCount.java *TopWordMapper. TopSumReducer.j TopWo

```java
package stubs;
import java.io.IOException;
public class TopWordMapper extends Mapper<Object, Text, IntWritable, IntWritable> {
    private IntWritable outKey = new IntWritable();
    /**
     * TopWordMapper reads the output of a previous word count MapReduce job,
     * extracts the frequency (count) of each word, and emits (count, 1) pairs.
     */
  @Override
  public void map(Object key, Text value, Context context)
      throws IOException, InterruptedException {
    // Split the input line into two parts: [word, count]
      String[] parts = value.toString().split("\\s+");
      if (parts.length == 2) {
          try {
            // Parse the second part as an integer count
              int count = Integer.parseInt(parts[1]);
           // Emit the count as key, and 1 as the value
              outKey.set(count);
              context.write(outKey, new IntWritable(1));
          } catch (NumberFormatException e) {
              // Ignore malformed lines
          }
      }
  }
```

Figure 9. The mapper class for parsing (word, count) lines and emits (count, 1).

TopWordMapper.j *WordMapper.jav SumReducer.java WordCount.java TopWordMapper.j TopSumReducer.j

```java
package stubs;

import java.io.IOException;

/**
 * TopSumReducer counts how many words occur exactly once in the entire dataset.
 * Process frequency keys and sums how many times each frequency appears.
 */
public class TopSumReducer extends Reducer<IntWritable, IntWritable, IntWritable, IntWritable> {
    // Counter to store the number of words that appear exactly once
    private int totalSingleCount = 0;

  @Override
    public void reduce(IntWritable key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
      // Only focus on frequency = 1
      if (key.get() == 1) {
          for (IntWritable val : values) {
              totalSingleCount += val.get();
          }
      }
    }

  @Override
  protected void cleanup(Context context) throws IOException, InterruptedException {
      // Emit the result only once after all reduce calls are done
      context.write(new IntWritable(1), new IntWritable(totalSingleCount));
  }
}

```

Figure 10. The reducer class to sum up the number of words with count = 1.

4. List all commands used in this assignment in order, as well as readable execution screenshots with your name and student ID printed from the driver code. **[15 mark]**

To begin, the Java classes are compiled by first navigating to their directory using the command "*cd workspace/Assignment1/src/stubs*". The command "*ls -alh*" is then used to verify that the required Java files are present in the directory. After confirming, the command "*cd ../*" is executed to return to the parent folder in preparation for the compilation step.

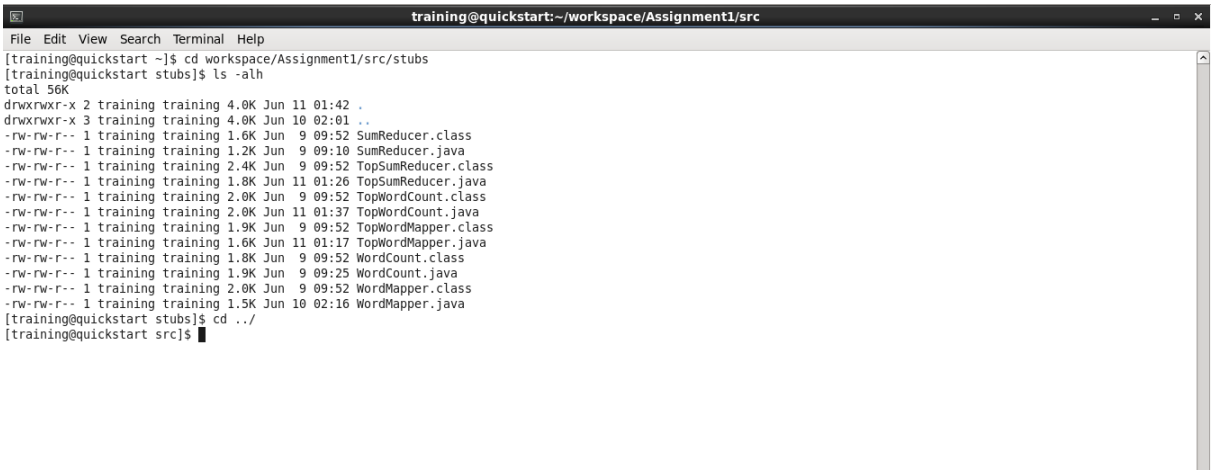

Figure 11. The commands for navigating and checking the Java classes.

Once inside the correct directory, the Java classes located in the "*stubs*" folder are compiled using the command "*javac -classpath `hadoop classpath` stubs/*.java*".

Figure 12. The command for compiling the Java classes.

After successful compilation, the command "*jar cvf wca1.jar stubs/*.class*" is used to package all the compiled Java class files from the "*stubs*" folder into a JAR file named "*wca1.jar*".

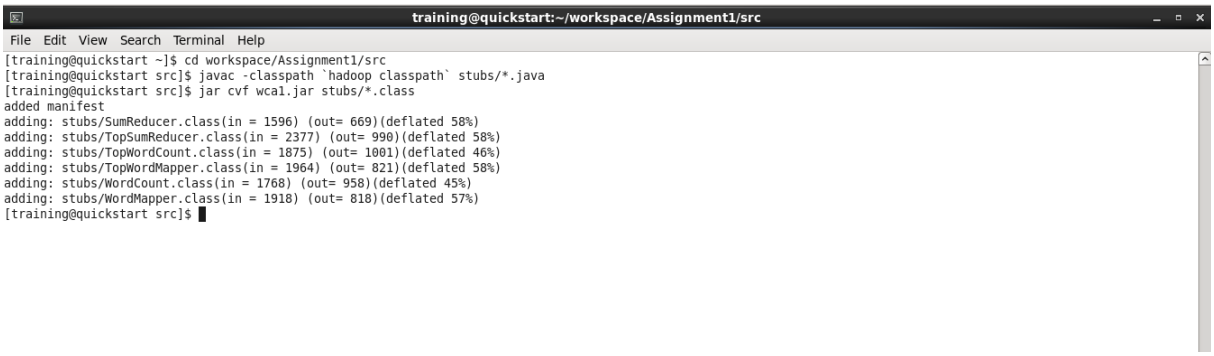

Figure 13. The command for collecting compiled Java files into a JAR file.

Prior to running the MapReduce job, the dataset needs to be uploaded to HDFS. This is done by first navigating to the directory containing the dataset using the command "*cd workspace/2025_assignment1*", and then executing "*hdfs dfs -put austlang_dataset.csv*" to transfer the dataset into the HDFS.

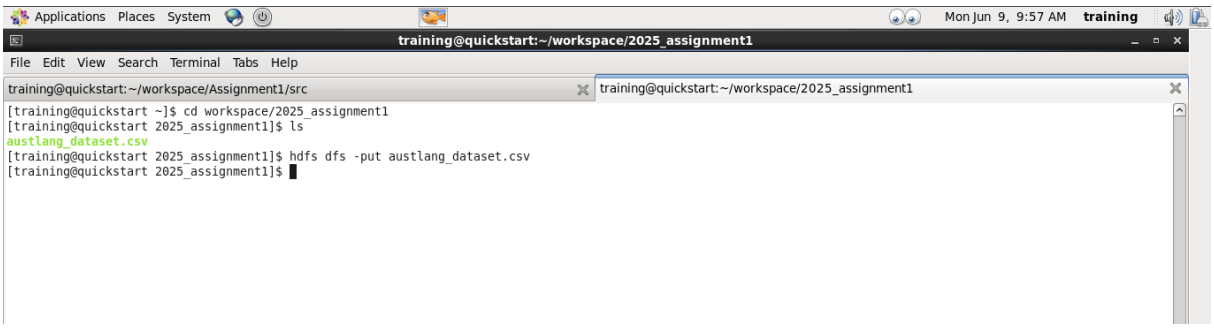

Figure 14. The commands for putting the dataset into HDFS.

Once the dataset has been successfully uploaded to HDFS, navigate back to the directory containing the previously created JAR file. Then, initiate the first MapReduce job "***WordCount***" using the uploaded dataset as input and specify "*wordcountsa1*" as the name of the output directory.

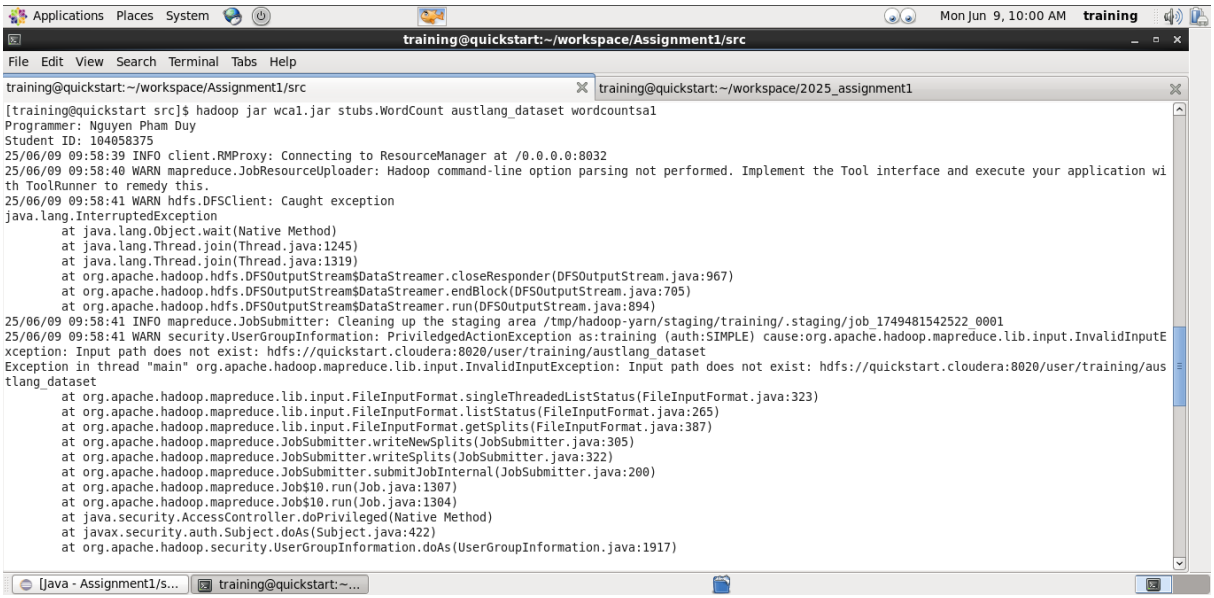

Figure 15. The command for executing the first MapReduce job.

After the execution is complete, retrieve the output using the command "*hdfs dfs -get*

wordcountsa1", and then examine the results of the first MapReduce job by running "*less wordcountsa1/part-r-00000*".

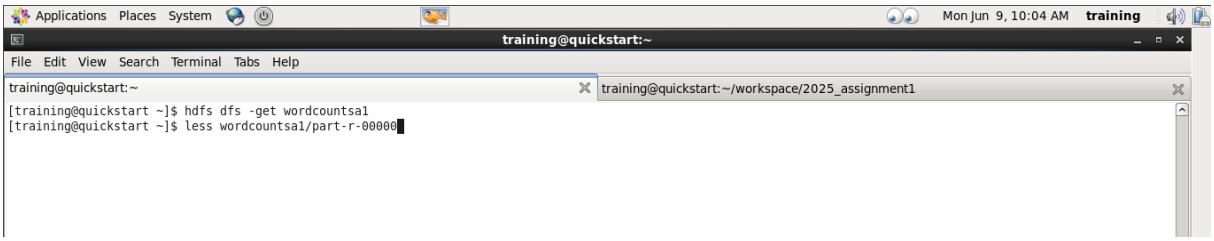

Figure 16. The commands for harvesting and checking the result of the first MapReduce job.

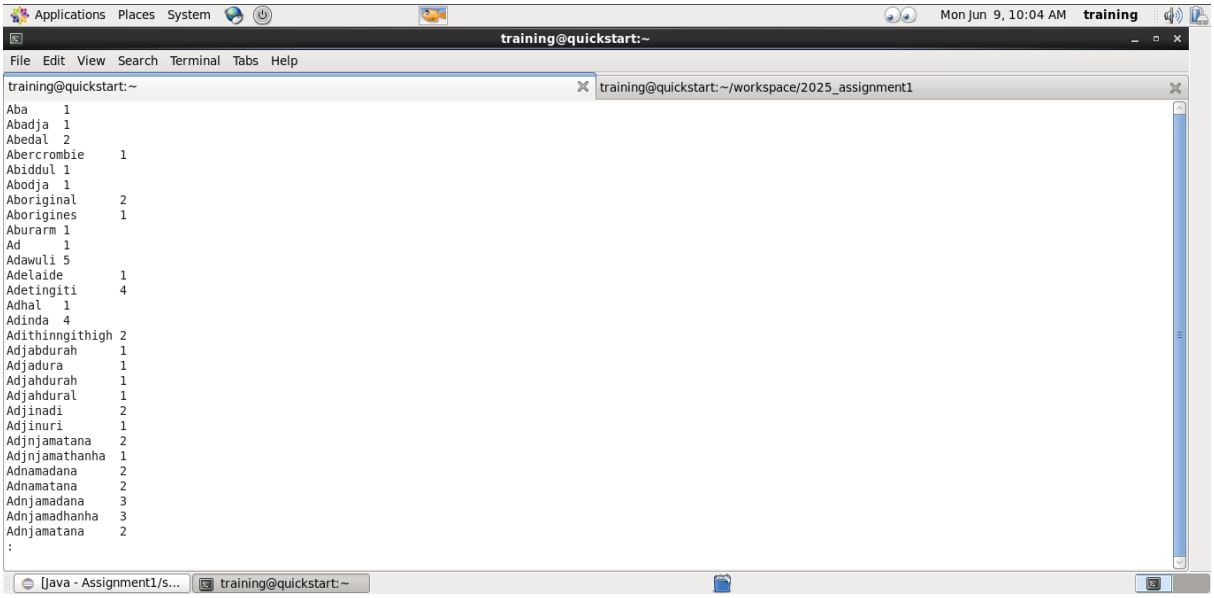

Figure 17. The result of the first MapReduce job.

Similar to the first MapReduce job, the second job is executed using the command "*hadoop jar wca1.jar stubs/TopWordCount wordcounta1/part-r-00000 topwordcountsa1*". This runs the "***TopWordCount***" program, using the output from the first job (wordcounta1/part-r-00000) as input, and stores the results in a new output directory named "*topwordcountsa1*".

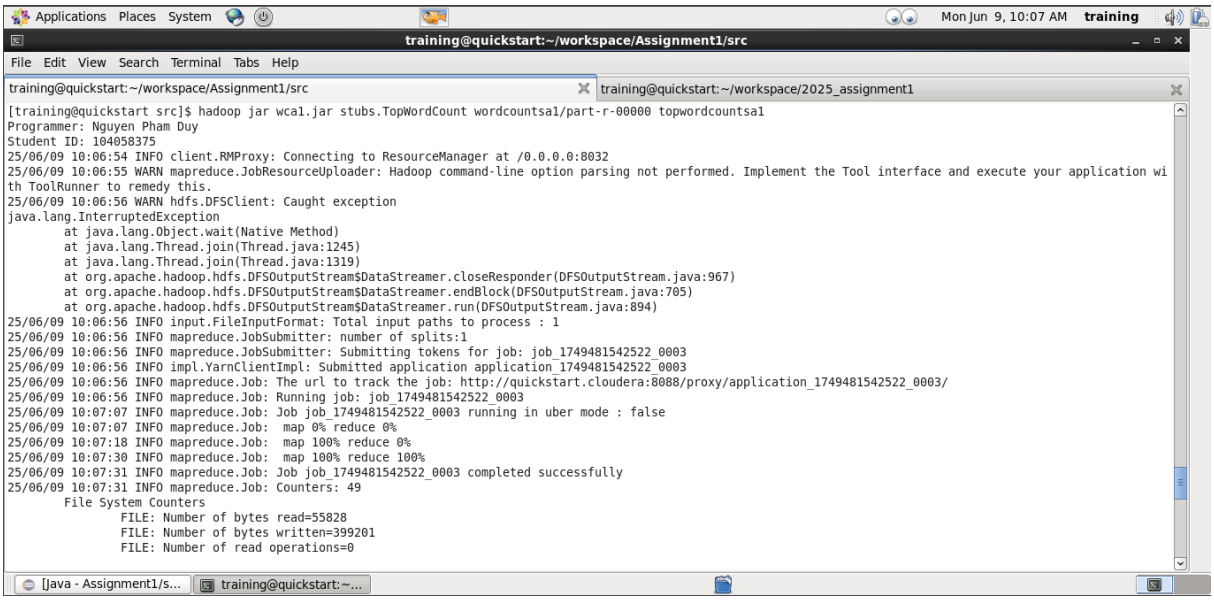

Figure 18. The command for executing the second MapReduce job.

Once the second MapReduce job has completed successfully, retrieve the output using the

command "*hdfs dfs -get topwordcountsa1*", and view the results by running "*less topwordcountsa1/part-r-00000*".

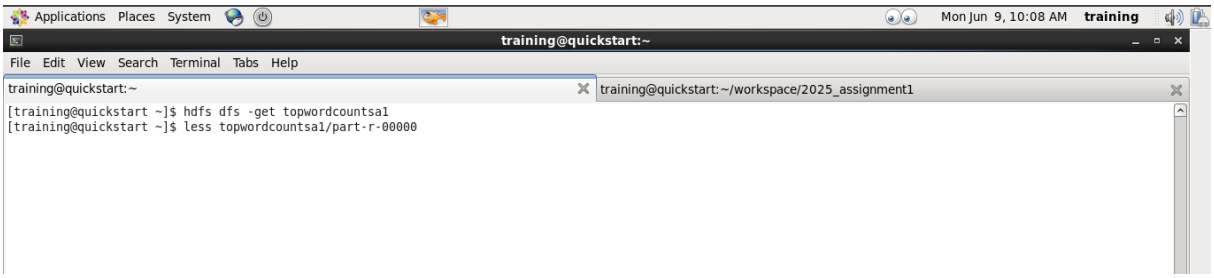

Figure 19. The commands for harvesting and collecting the result of the second MapReduce job.

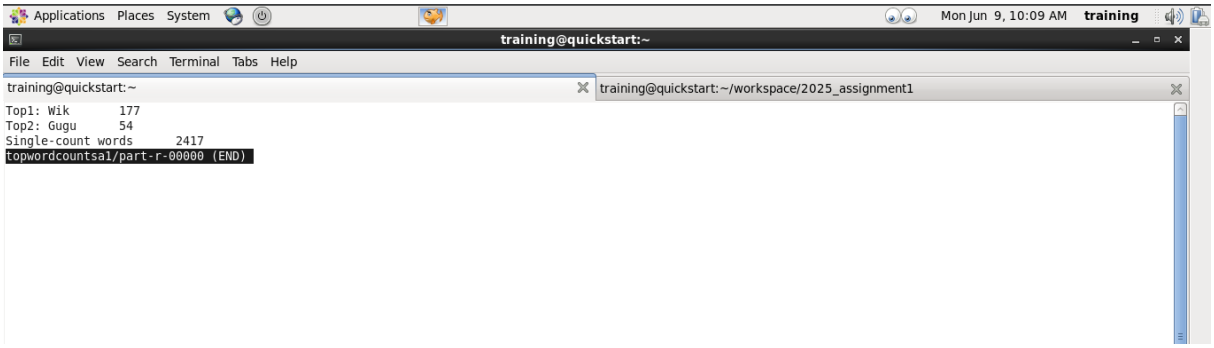

Figure 20. The result of the second MapReduce job.

(*)Additional MapReduce Implementation: Counting Terms that start with all upper-case letters and follow by the lower-case letters Appearing Only Once in the Entire Document:

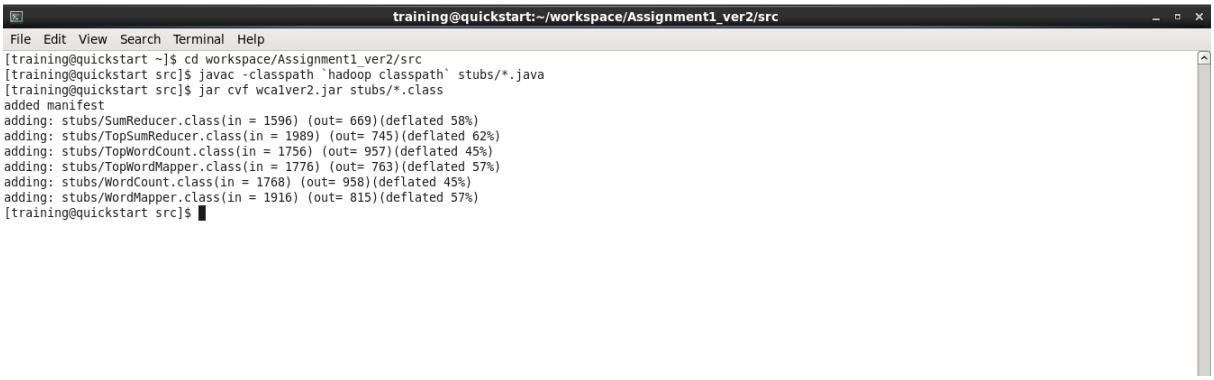

Figure 21. The commands to compile and collect the compiled Java files into a JAR file.

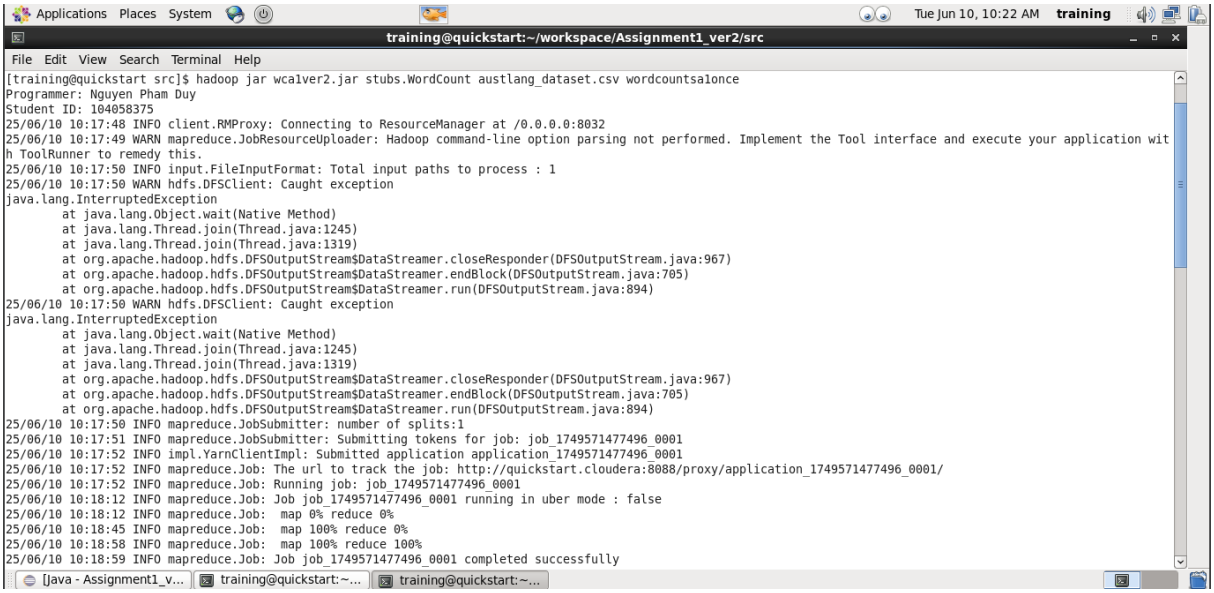

Figure 22. The command for executing the first MapReduce on Hadoop.

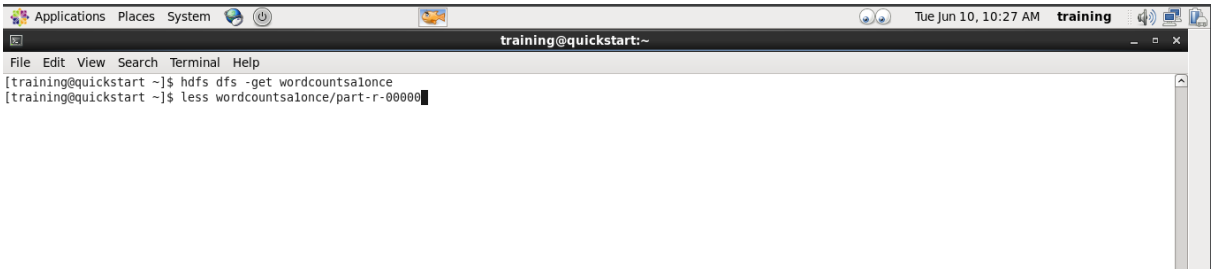

Figure 23. The commands for harvesting and checking the result of the first MapReduce.

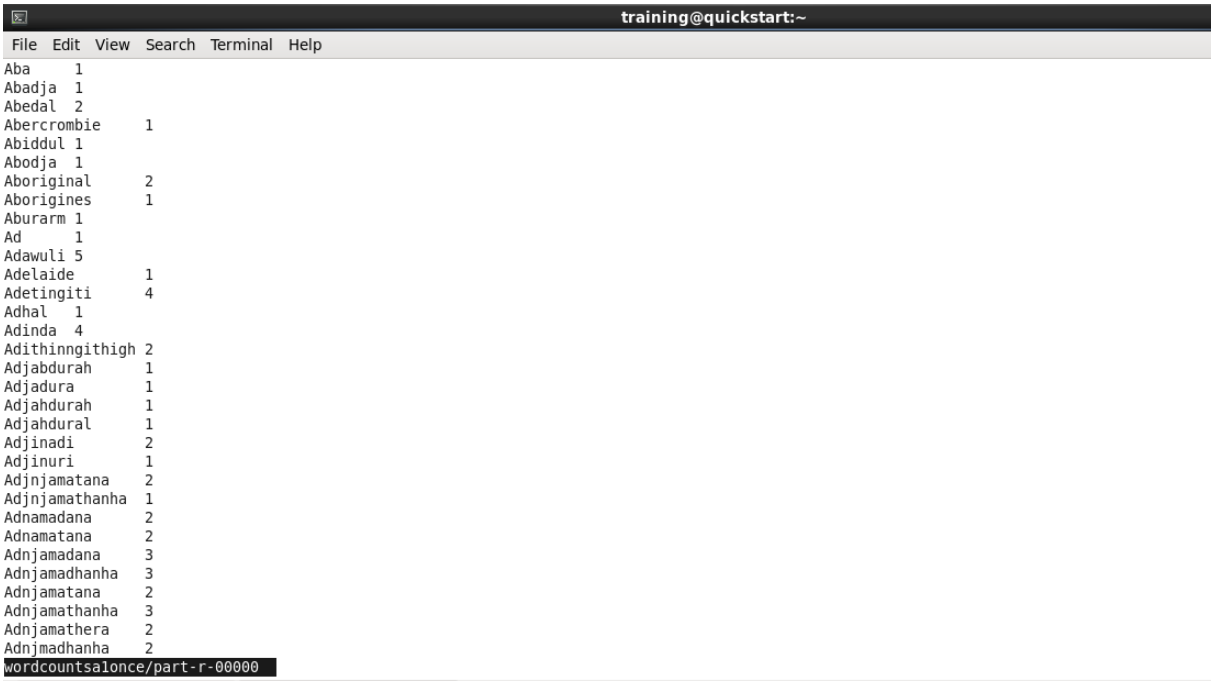

Figure 24. The result of the first MapReduce.

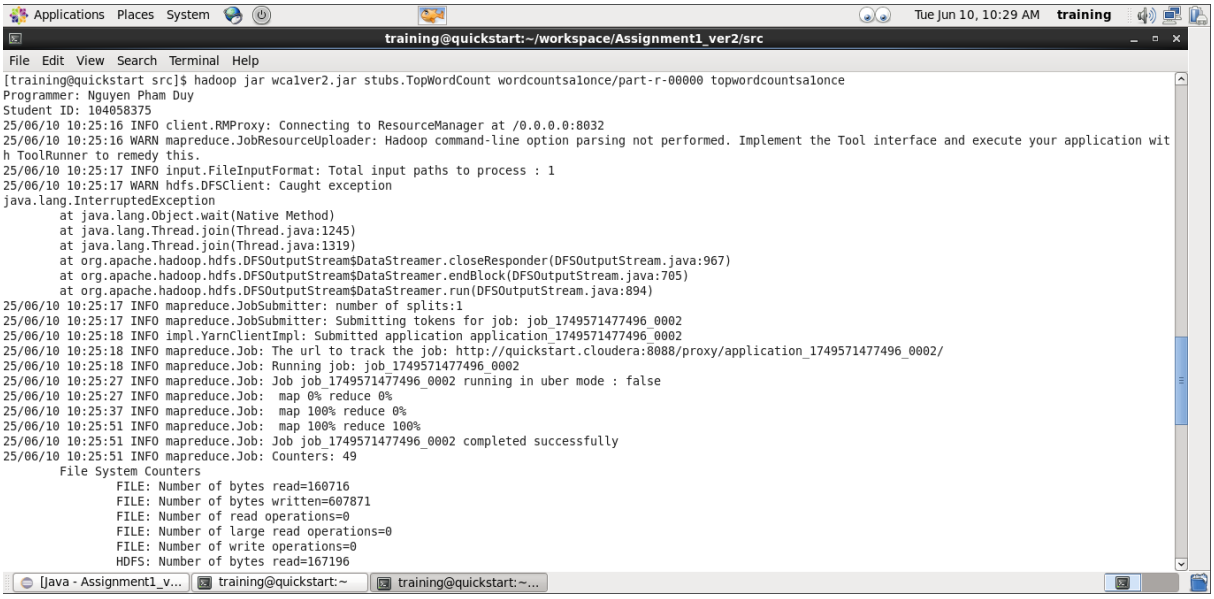

Figure 25. The command for executing the second MapReduce on Hadoop.

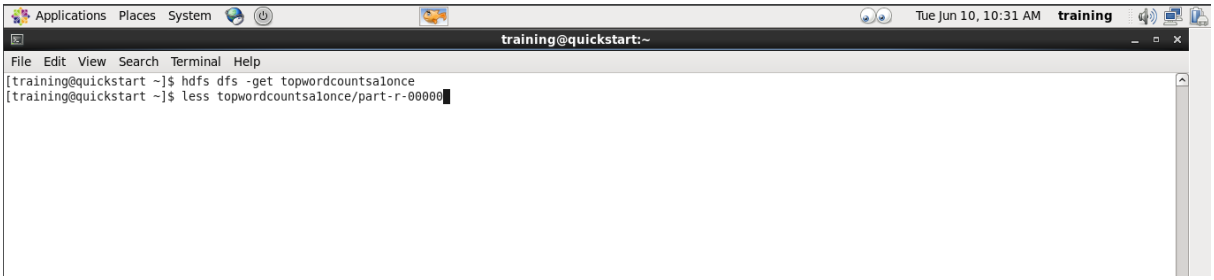

Figure 26. The commands for harvesting and checking the result of the second MapReduce.

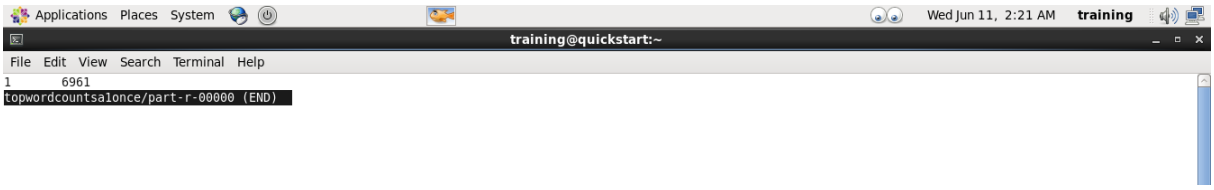

Figure 27. The result of the second MapReduce.

5. You are assigned to find the word counts of any terms in the given dataset starting with the upper-case letter "A," "C," "G," "W," or "Y," followed by lowercase letters regardless of the length. Answering the questions:

   a. What is the term under the given criteria with the highest appearance count, and what is its count? **[10 marks]**

      The term is **"Wik",** and its appearance count is **177**.

   b. What is the term under the given criteria for the second high appearance count, and what is its count? **[10 marks]**

      The term is **"Gugu",** and its appearance count is **54**.

   c. How many terms only have a single count in the document? **[5 marks]**

There are **2417** terms that have a single count in the document under the given criteria, and there are **6961** terms only have a single count for all words start with a upper-case letter and follow by the lower-case letters in the whole document.

6. Video presentation with the required content. **[Pass/Fail]**

   The video needs to be submitted independently on Canvas.